

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №2
дисциплины
«Искусственный интеллект и машинное обучение»

Выполнил:
Лебединский Даниил
Александрович 2 курс,
группа ИВТ-б-о-24-2,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверил:
Доцент департамента цифровых,
робототехнических систем и
электроники института
перспективной инженерии Воронкин
Р.А

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Цель работы: исследовать базовые возможности библиотеки NumPy языка программирования Python.

Ссылка на репозиторий: <https://github.com/FomalhautSystem/AI-LAB-2/tree/main>

Конспект теоретического материала:

Библиотека NumPy

NumPy (от Numerical Python) — это открытая библиотека для Python, созданная для эффективной работы с многомерными массивами и матрицами. Она предоставляет широкий набор высокоуровневых математических функций для обработки таких структур данных и выполнения вычислений. NumPy — одна из самых востребованных библиотек в научных и инженерных расчётах, а также базовая составляющая многих других инструментов Python, включая SciPy, Pandas и TensorFlow. Главная структура данных в NumPy — многомерный массив `ndarray`. Это объект, представляющий таблицу данных (например, вектор или матрицу) и обеспечивающий высокую эффективность операций над ними.

Одномерные массивы (вектора) — применяются для представления наборов данных.

Двумерные массивы (матрицы) — широко используются в линейной алгебре и машинном обучении.

Многомерные массивы (тензоры) — важны в глубоких нейронных сетях и других сложных вычислениях.

Ход работы:

В ходе работы были выполнены практические задания.

1. Создайте массив NumPy размером 3x3, содержащий числа от 1 до 9. Умножьте все элементы массива на 2, а затем замените все элементы больше 10 на 0. Выведите итоговый массив.

Решение:

```
import numpy as np

mas = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])
mas = 2 * mas
mas[mas < 10] = 0
print(mas)
```

Результат выполнения программы:

```
[[ 0  0  0]
 [ 0 10 12]
 [14 16 18]]
```

2. Создайте массив NumPy из 20 случайных целых чисел от 1 до 100. Найдите и выведите все элементы, которые делятся на 5 без остатка. Затем замените их на -1 и выведите обновленный массив.

Решение:

```
import numpy as np

mas = np.random.randint(1, 101, size = 20)
print(mas)
print(mas[mas % 5 == 0])
mas[mas % 5 == 0] = -1
print(mas)
```

Результат выполнения программы:

```
[ 2 46 72 59  1 89 42 94 74 47 65 55 39 22 92 72 65 26 83 38]
[65 55 65]
[ 2 46 72 59  1 89 42 94 74 47 -1 -1 39 22 92 72 -1 26 83 38]
```

3. Создайте два массива NumPy размером 1x5, заполненные случайными числами от 0 до 50.

- Объедините эти массивы в один двумерный массив (по строкам).
- Разделите полученный массив на два массива, каждый из которых содержит 5 элементов.

Выведите все промежуточные и итоговые результаты.

Решение:

```

import numpy as np

mas1 = np.random.randint(0, 51, size=(1, 5))
mas2 = np.random.randint(0, 51, size=(1, 5))

print(mas1)
print(mas2)

mas = np.concatenate((mas1, mas2), axis=0)
print(mas)

split1, split2 = np.vsplit(mas, 2)

print("Первый разделенный массив")
print(split1)
print("Второй разделенный массив")
print(split2)

```

Результат выполнения программы:

```

[[10 40 14 36 34]]
[[49 42  4 33 24]]
[[10 40 14 36 34]
 [49 42  4 33 24]]
Первый разделенный массив
[[10 40 14 36 34]]
Второй разделенный массив
[[49 42  4 33 24]]

```

4. Создайте массив из 50 чисел, равномерно распределенных от -10 до 10. Вычислите сумму всех элементов, сумму положительных элементов и сумму отрицательных элементов. Выведите результаты.

Решение:

```

import numpy as np

arr = np.linspace(-10, 10, 50)

print("Массив из 50 чисел от -10 до 10:")
print(arr)

summa = np.sum(arr)
print(f"Сумма всех элементов: {summa}")

polozh_summa = np.sum(arr[arr > 0])
print(f"Сумма положительных элементов: {polozh_summa}")

otriza_sum = np.sum(arr[arr < 0])
print(f"Сумма отрицательных элементов: {otriza_sum}")

```

Результат выполнения программы:

Массив из 50 чисел от -10 до 10:

```
[-10.      -9.59183673  -9.18367347  -8.7755102   -8.36734694
 -7.95918367  -7.55102041  -7.14285714  -6.73469388  -6.32653061
 -5.91836735  -5.51020408  -5.10204082  -4.69387755  -4.28571429
 -3.87755102  -3.46938776  -3.06122449  -2.65306122  -2.24489796
 -1.83673469  -1.42857143  -1.02040816  -0.6122449   -0.20408163
  0.20408163  0.6122449    1.02040816  1.42857143  1.83673469
  2.24489796  2.65306122  3.06122449  3.46938776  3.87755102
  4.28571429  4.69387755  5.10204082  5.51020408  5.91836735
  6.32653061  6.73469388  7.14285714  7.55102041  7.95918367
  8.36734694  8.7755102   9.18367347  9.59183673 10.      ]
```

Сумма всех элементов: 7.105427357601002e-15

Сумма положительных элементов: 127.55102040816328

Сумма отрицательных элементов: -127.55102040816327

5. Создайте:

- Единичную матрицу размером 4x4.
- Диагональную матрицу размером 4x4 с диагональными элементами [5, 10, 15, 20] (не использовать циклы).

Найдите сумму всех элементов каждой из этих матриц и сравните результаты.

Решение:

```
import numpy as np

ones_matrix = np.eye(4)
print(ones_matrix)

diagonal = [5, 10, 15, 20]
diagonal_matrix = np.diag(diagonal)
print(diagonal_matrix)

sum_ones = np.sum(ones_matrix)
sum_diagonal = np.sum(diagonal_matrix)

print(f"Сумма всех элементов единичной матрицы: {sum_ones}")
print(f"Сумма всех элементов диагональной матрицы: {sum_diagonal}")
print(f"Разница между суммами: {abs(sum_diagonal - sum_ones)}")
```

Результат выполнения программы:

```
[[1.  0.  0.  0.]
 [0.  1.  0.  0.]
 [0.  0.  1.  0.]
 [0.  0.  0.  1.]]
[[ 5  0  0  0]
 [ 0 10  0  0]
 [ 0  0 15  0]
 [ 0  0  0 20]]
Сумма всех элементов единичной матрицы: 4.0
Сумма всех элементов диагональной матрицы: 50
Разница между суммами: 46.0
```

6. Создайте две квадратные матрицы NumPy размером 3x3, заполненные случайными целыми числами от 1 до 20. Вычислите и выведите:

- Их сумму
- Их разность
- Их поэлементное произведение

Решение:

```
import numpy as np

matrix1 = np.random.randint(1, 21, size=(3, 3))
matrix2 = np.random.randint(1, 21, size=(3, 3))

print(matrix1)
print(matrix2)

sum_matrix = matrix1 + matrix2
print("\nСумма матриц (A + B):")
print(sum_matrix)

razn_matrix = matrix1 - matrix2
print("\nРазность матриц (A - B):")
print(razn_matrix)

# Вычисляем поэлементное произведение
proizv = matrix1 * matrix2
print("\nПоэлементное произведение (A * B):")
print(proizv)
```

Результат выполнения программы:

```
[[11 15 12]
 [16  1  8]
 [ 6 17 19]]
[[10  3  2]
 [12 14  9]
 [19 18  5]]
```

Сумма матриц (A + B):

```
[[21 18 14]
 [28 15 17]
 [25 35 24]]
```

Разность матриц (A - B):

```
[[ 1 12 10]
 [ 4 -13 -1]
 [-13 -1 14]]
```

Поэлементное произведение (A * B):

```
[[110 45 24]
 [192 14 72]
 [114 306 95]]
```

7. Создайте две матрицы NumPy:

- Первую размером 2x3, заполненную случайными числами от 1 до 10.
- Вторую размером 3x2, заполненную случайными числами от 1 до 10.

Выполните матричное умножение (@ или pr.dot) и выведите результат.

Решение:

```
import numpy as np

matrix1 = np.random.randint(1, 11, size=(2, 3))
print("Первая матрица (2x3):")
print(matrix1)

matrix2 = np.random.randint(1, 11, size=(3, 2))
print("\nВторая матрица (3x2):")
print(matrix2)

result = matrix1 @ matrix2
print("\nРезультат матричного умножения (2x3 x 3x2 = 2x2):")
print(result)
```

Результат выполнения программы:

Первая матрица (2x3):

```
[[3 3 4]
 [6 1 7]]
```

Вторая матрица (3x2):

```
[[10 10]
 [ 9  5]
 [ 4  7]]
```

Результат матричного умножения (2x3 x 3x2 = 2x2):

```
[[ 73  73]
 [ 97 114]]
```

8. Создайте случайную квадратную матрицу 3x3. Найдите и выведите:

Определитель этой матрицы

Обратную матрицу (если существует, иначе выведите сообщение, что матрица вырождена)

Используйте функции np.linalg.det и np.linalg.inv .

Решение:

```

import numpy as np

matrix = np.random.randint(1, 11, size=(3, 3))

print("Случайная матрица 3x3:")
print(matrix)

det = np.linalg.det(matrix)

if abs(det) != 0:
    inv_matrix = np.linalg.inv(matrix)
    print("\nОбратная матрица существует:")
    print(inv_matrix)

    prover = matrix @ inv_matrix
    print("\nПроверка:  $A \times A^{-1} = E$  (единичная матрица):")
    print(np.round(prover, 2))
else:
    print("\nМатрица вырождена (определитель = 0), обратная матрица не существует!")

```

Результат выполнения программы:

Случайная матрица 3x3:

```

[[8 7 8]
 [9 1 3]
 [3 2 7]]

```

Обратная матрица существует:

```

[[-0.004  0.132 -0.052]
 [ 0.216 -0.128 -0.192]
 [-0.06  -0.02  0.22 ]]

```

Проверка: $A \times A^{-1} = E$ (единичная матрица):

```

[[ 1.  0.  0.]
 [ 0.  1.  0.]
 [-0. -0.  1.]]

```

9. Создайте матрицу NumPy размером 4x4, содержащую случайные целые числа от 1 до 50. Выведите:

- Исходную матрицу
- Транспонированную матрицу
- След матрицы (сумму элементов на главной диагонали)

Используйте `np.trace` для нахождения следа.

Решение:

```

import numpy as np

matrix = np.random.randint(1, 51, size=(4, 4))

```

```

print("Исходная матрица (4x4):")
print(matrix)

transpon_matrix = matrix.T
print("\nТранспонированная матрица:")
print(transpon_matrix)

trace = np.trace(matrix)
print(f"\nСлед матрицы (сумма элементов на главной диагонали): {trace}")

```

Результат выполнения программы:

Исходная матрица (4x4):

```

[[ 8 14 30 10]
 [28 36  3 48]
 [ 5 46 50 23]
 [14 29 36 13]]

```

Транспонированная матрица:

```

[[ 8 28  5 14]
 [14 36 46 29]
 [30  3 50 36]
 [10 48 23 13]]

```

След матрицы (сумма элементов на главной диагонали): 107

10. Решить систему линейных уравнений вида:

$$\begin{cases} 2x + 3y - z = 5 \\ 4x - y + 2z = 6 \\ -3x + 5y + 4z = -2 \end{cases}$$

Используйте матричное представление $Ax = B$, где A – матрица коэффициентов, x – вектор неизвестных, b – вектор правой части. Решите систему с помощью `np.linalg.solve` и выведите результат.

Решение:

```

import numpy as np

A = np.array([
    [2, 3, -1],
    [4, -1, 2],
    [-3, 5, 4]
])

B = np.array([5, 6, -2])

print("Система линейных уравнений:")
print("2x + 3y - z = 5")
print("4x - y + 2z = 6")
print("-3x + 5y + 4z = -2")

print("Матрица коэффициентов A:")
print(A)

```

```

print()
print("Вектор правой части B:")
print(B)
print()

resh = np.linalg.solve(A, B)

print("Решение системы:")
print(f"x = {resh[0]}")
print(f"y = {resh[1]}")
print(f"z = {resh[2]}")

```

Результат выполнения программы:

Система линейных уравнений:

$$2x + 3y - z = 5$$

$$4x - y + 2z = 6$$

$$-3x + 5y + 4z = -2$$

Матрица коэффициентов A:

```

[[ 2  3 -1]
 [ 4 -1  2]
 [-3  5  4]]

```

Вектор правой части B:

```
[ 5  6 -2]
```

Решение системы:

$$x = 1.6396396396396398$$

$$y = 0.5765765765765767$$

$$z = 0.009009009009008976$$

Индивидуальное задание:

Решите индивидуальное задание согласно варианту. Каждое задание предусматривает построение системы линейных уравнений. Решите полученную систему уравнений с использованием библиотеки NumPy. Для решения системы используйте метод Крамера и матричный метод. Сравните полученные результаты, с результатами, полученными с помощью `np.linalg.solve`. Перераспределение бюджета. Городской бюджет распределяется между образованием, здравоохранением и транспортом. На здравоохранение выделяется в 1.5 раза больше, чем на транспорт, а на образование - на 20% больше, чем на здравоохранение. Всего в бюджете 5 000 000 рублей. Сколько денег выделено на каждую сферу?

Решение:

```

import numpy as np

A = np.array([[-1.5, 1, 0],
              [0, -1.2, 1],
              [1, 1, 1]])

```

```

B = np.array([0, 0, 5_000_000])

matr = np.linalg.inv(A) @ B

det_A = np.linalg.det(A)

def kram(A, B, col):
    A_i = A.copy()
    A_i[:, col] = B
    return np.linalg.det(A_i) / det_A

kramer = np.array([kram(A, B, i) for i in range(3)])

solve = np.linalg.solve(A, B)

print("Матричный метод:", matr)
print("Метод Крамера: ", kramer)
print("np.linalg.solve: ", solve)

```

Результат выполнения программы:

```

Матричный метод: [1162790.69767442 1744186.04651163 2093023.25581395]
Метод Крамера:   [1162790.69767442 1744186.04651163 2093023.25581396]
np.linalg.solve: [1162790.69767442 1744186.04651163 2093023.25581395]

```

Контрольные вопросы:

1. Каково назначение библиотеки NumPy?

NumPy нужна для эффективной работы с большими массивами чисел и предоставляет высокоуровневые математические функции для линейной алгебры, преобразования Фурье и генерации случайных чисел. Она лежит в основе многих научных пакетов Python, обеспечивая быстрый и удобный доступ к элементам массива, векторизацию операций и эффективные алгоритмы численных расчётов.

2. Что такое массивы ndarray?

ndarray (n-dimensional array) — основной объект библиотеки NumPy, представляющий N-мерные однородные массивы фиксированного размера. Они позволяют эффективно обрабатывать большие объёмы данных за счёт оптимизации на уровне C и поддерживают быстрые операции с элементами: арифметику, срезы, транспонирование и множество научных и математических функций.

3. Как осуществляется доступ к частям многомерного массива?

Доступ к частям многомерного массива в NumPy выполняется через

индексирование и нарезание (slicing). Индексация — указание координат элемента в квадратных скобках, начиная с нуля. Нарезка задаётся диапазоном индексов в формате [start:stop:step] и позволяет извлекать подмножества элементов. Например, для первого столбца двумерного массива используется запись `array[:, 0]`.

4. Как осуществляется расчёт статистик по данным?

Расчёт статистик в NumPy выполняется специализированными методами объектов `ndarray`: `mean()` — среднее, `std()` — стандартное отклонение, `min()` и `max()` — минимум и максимум, `sum()` — сумма, `prod()` — произведение, `argmin()` и `argmax()` — индексы минимального и максимального значений. Эти методы принимают аргумент `axis`, задающий ось расчёта (по строкам или столбцам).

5. Как выполняется выборка данных из массивов `ndarray`?

Выборка данных из `ndarray` выполняется через индексацию и нарезку. Можно выбирать элементы по отдельным индексам или диапазонам, используя квадратные скобки. Например, `arr[1, 2]` возвращает элемент из второй строки и третьего столбца. Для сложных условий применяется булево индексирование: `arr[arr > threshold]` вернёт элементы, превышающие порог. Также поддерживается выборка целыми массивами индексов: `arr[[0, 2], :]` выберет первую и третью строки.

6. Приведите основные виды матриц и векторов. Опишите способы их создания в языке Python.

Матрицы и векторы — структуры данных для представления двумерных и одномерных массивов соответственно. Основные виды матриц: единичные, диагональные, симметричные, ортогональные и треугольные. Векторы бывают скалярные (одномерные) и вектор-столбец (двумерные с одним столбцом).

```
import numpy as np
vector = np.array([1, 2, 3])
```

```
matrix = np.array([[1, 2, 3], [4, 5, 6]])  
identity_matrix = np.eye(3)  
diagonal_matrix = np.diag([1, 2, 3])  
random_matrix = np.random.rand(3, 3)
```

7. Как выполняется транспонирование матриц?

Транспонирование выполняется методом `.T` объекта `ndarray` или функцией `numpy.transpose()`. Транспонированная матрица получается заменой строк на столбцы исходной.

8. Приведите свойства операции транспонирования матриц.

Ассоциативность относительно умножения, дистрибутивность относительно суммы, идемпотентность, умножение скаляра на матрицу.

9. Какие имеются средства в библиотеке NumPy для выполнения транспонирования матриц?

Метод `.T`, функция `numpy.transpose()`, аргумент `axes`.

10. Какие существуют основные действия над матрицами?

Сумма, вычитание, умножение, умножение на число, транспонирование, определитель, обращение, собственные числа и собственные векторы.

11. Как осуществляется умножение матрицы на число?

```
matrix = np.array([[1, 2],  
[3, 4]])  
result = matrix * 2
```

12. Какие свойства операции умножения матрицы на число?

Ассоциативность, дистрибутивность относительно сложения матриц и чисел.

13. Как осуществляются операции сложения и вычитания матриц?

```
A = np.array([[1, 2], [3, 4]])
```

```
B = np.array([[5, 6], [7, 8]])
```

```
C = A + B
```

```
D = A - B
```

14. Каковы свойства операций сложения и вычитания матриц?

Коммутативность, ассоциативность, наличие нулевой матрицы.

15. Какие имеются средства в библиотеке NumPy для выполнения операций сложения и вычитания матриц?

Операторы + и -, функции add() и subtract().

16. Как осуществляется операция умножения матриц?

```
A = np.array([[1, 2], [3, 4]])
```

```
B = np.array([[5, 6], [7, 8]])
```

```
C = np.dot(A, B)
```

17. Каковы свойства операции умножения матриц?

Ассоциативность, дистрибутивность относительно сложения, некоммутативность.

18. Какие имеются средства в библиотеке NumPy для выполнения операции умножения матриц?

numpy.dot(), оператор @, numpy.matmul().

```
C = A @ B
```

19. Что такое определитель матрицы?

Определитель матрицы — специальное вещественное число, которое сопоставляется каждой квадратной матрице и описывает её важные геометрические и алгебраические характеристики: изменение объёма при линейном преобразовании, наличие обратной матрицы, решение систем линейных уравнений.

20. Какие имеются средства в библиотеке NumPy для нахождения значения определителя матрицы?

`linalg.det()`.

21. Что такое обратная матрица? Какой алгоритм нахождения обратной матрицы?

Алгоритм нахождения обратной матрицы основан на методах линейной алгебры: метод Гаусса-Жордана, LU-разложение или SVD-разложение. Наиболее распространён метод LU-разложения (разложение на нижнюю и верхнюю треугольные матрицы), после которого находят обратную матрицу.

22. Каковы свойства обратной матрицы?

Единственность, обратная к обратной, перестановочность с транспонированием.

23. Какие имеются средства в библиотеке NumPy для нахождения обратной матрицы?

```
A = np.array([[1, 2], [3, 4]])  
inverse_A = np.linalg.inv(A)
```

24. Изучить метод Крамера для решения СЛУ. Привести алгоритм решения СЛУ методом Крамера средствами библиотеки NumPy.

1. Найти определитель исходной матрицы A .
2. Если $\det(A) \neq 0$, система имеет единственное решение.
3. Подставляя вектор B на место каждого столбца матрицы A , вычислить вспомогательные определители $\text{del}(i)$.
4. $X_i = \text{del}_i / \text{del}$, $i = 1 \dots n$, где del — определитель исходной матрицы, del_i — определитель матрицы, полученной заменой i -го столбца на вектор B .

25. Изучить матричный метод для решения СЛУ. Привести алгоритм решения СЛУ матричным методом средствами библиотеки NumPy.

Метод сводится к приведению системы к виду $AX = B$, где A — матрица

коэффициентов, X — вектор неизвестных, B — вектор правых частей. Суть метода:

1. Проверяется условие разрешимости: матрица A должна быть невырожденной, то есть иметь ненулевой определитель.
2. Находится обратная матрица.
3. Искомое решение: $X = A^{-1}B$.

Вывод: В ходе выполнения работы были исследованы базовые возможности библиотеки NumPy, необходимые для решения систем линейных уравнений, включая создание многомерных массивов с помощью `np.array()`, вычисление определителя матрицы через `np.linalg.det()`, нахождение обратной матрицы посредством `np.linalg.inv()`, матричное умножение оператором `@`, прямое решение систем линейных уравнений функцией `np.linalg.solve()`, а также реализацию метода Крамера путём замены столбцов матрицы. В результате все три способа решения системы (матричный метод, метод Крамера и встроенная функция `np.linalg.solve()`) дали идентичные результаты с высокой точностью, что подтверждает корректность работы NumPy и эффективность его использования для задач линейной алгебры.